06 iDeliveryFood Todos Prompts

Regras desta revisão — aplicação integral

Este documento conserva o conteúdo e os módulos do PDF anexado, com alterações nas instruções afetadas, sem resumo. As regras abaixo se aplicam a todos os módulos e critérios de aceite deste arquivo.

Mobile nativo e testes por plataforma

Android: Kotlin e Jetpack Compose; iOS: Swift e SwiftUI, projetos separados. Sem app híbrido/React. Implementar e testar cada plataforma independentemente: unitários de domínio e ViewModel, integração API, UI, deep links, sessão, permissões, offline e acessibilidade. CI Android com toolchain apropriado; CI iOS em runner macOS com Xcode. Assinatura e contas de distribuição são externas ao Git. Falta de ambiente Apple deve aparecer como pendência, nunca como build/teste concluído. Web/PWA permanece entrega web separada, jamais substitui os apps nativos.

MCP exato fornecido pelo proprietário

{
"mcpServers": {
"supabase": {
"serverUrl": "https://mcp.supabase.com/mcp?project_ref=dwsoochnfxldfvcaijie&features=docs%2Caccount%2Cdatabase%2Cdebugging%2Cdevelopment%2Cfunctions%2Cbranching%2Cfeedback"
}
}
}

SQL remoto e seed

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

GitHub depois dos testes

O Antigravity deve executar commit e push em https://github.com/JuanBMMontesinos/iDeliveryFood.git depois de testar o código e revisar diff e segredos. Verificar branch/remotes e preservar histórico; não

usar force-push, mudar visibilidade ou contornar proteção. Respeitar gates de aprovação do ambiente e autenticar por mecanismo seguro. Executar testes locais antes do push e acompanhar CI remoto depois; falha de CI requer correção e novo push, sem alegar release aprovado. Confirmar SHA remoto. Push não é deploy nem publicação nas lojas.

Idioma principal

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

Documentação e segurança preservadas

Manter documentação completa, testes unitários, integração, RLS/pgTAP, e2e e controles de cybersegurança dos módulos originais. Atualizar arquitetura, ERD, migrations aplicadas, seed, API, matriz de permissões, instruções nativas de build/teste, i18n, CI/CD, operação e evidências. Não incluir segredos; não declarar certificação de segurança sem validação independente.

Conteúdo integral revisado do documento anexado

PACOTE REVISADO — PROMPTS POR APLICATIVO, PORTAL E MÓDULO

Este material substitui o pacote anterior. É uma especificação e conjunto de comandos para o agente construir o iDeliveryFood incrementalmente; não é código já implementado nem certificação de produção.

Os prompts cobrem explicitamente app web e app mobile do cliente, app mobile do entregador, portal do parceiro, gestor de pedidos, portal de redes, console logístico, área de frota, administração central e backoffices financeiro, atendimento, comercial, marketing, relatórios, risco e privacidade, além da infraestrutura comum.

Como usar: cole as instruções permanentes antes do prompt do módulo. Em uma nova sessão, repita-as e peça ao agente para ler docs/DELIVERY_STATUS.md e os contratos existentes. Execute um módulo por tarefa, respeitando dependências. Não envie todos para execução simultânea. IDs indicam módulos, não obrigatoriamente a ordem numérica.

Projeto Supabase corrigido: dwsoochnfxldfvcaijie. Executar migrations e seed no ambiente de desenvolvimento autorizado conforme esta revisão. Rotacionar a senha exposta; arquivos não contêm senha nem URI credenciada.

Não há publicação GitHub, acesso ao banco ou implantação realizada por este pacote. Contas externas, contratação de serviços, credenciais seguras, revisão jurídica e aprovações permanecem responsabilidade do proprietário.

INSTRUÇÕES PERMANENTES — COPIAR COMO CONTEXTO EM TODAS AS SESSÕES

Você é a equipe de engenharia do iDeliveryFood dentro do Antigravity. Execute uma etapa por vez e produza código funcional, não apenas planos, protótipos ou descrições. Inspecione o repositório e siga os contratos/ADRs existentes. Não substitua stack ou reescreva módulos prontos sem justificar e obter aprovação. Identifique esta entrega pelo ID do módulo, mantenha docs/DELIVERY_STATUS.md atualizado e não afirme conclusão sem evidências.

REPOSITÓRIO: alvo solicitado pelo proprietário: https://github.com/JuanBMMontesinos/iDeliveryFood.git. Verifique primeiro acesso, conteúdo, branch, remotes e estado local. Não suponha que exista ou esteja vazio. Após testes aprovados, executar commit e push ao repositório solicitado e aplicar migrations no Supabase autorizado. Respeitar aprovações do ambiente; não alterar visibilidade, usar force-push ou executar operações destrutivas sem autorização específica. Antes de qualquer publicação: revisar diff, testes e secret scan; não expor credenciais ou dados pessoais.

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação

específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

ARQUITETURA DE REFERÊNCIA: monorepo com backend/web TypeScript e workspace pnpm para apps/customer-web (Next.js), apps/partner-web (Next.js), apps/admin-web (Next.js), apps/api (NestJS) e apps/worker. Mobile com código NATIVO separado: apps/customer-android e apps/courier-android em Kotlin/Jetpack Compose; apps/customer-ios e apps/courier-ios em Swift/SwiftUI. Se algum novo app mobile for habilitado, também terá implementações Android e iOS independentes. NÃO usar React Native, Expo, Flutter, Ionic, Capacitor ou solução híbrida; não encapsular páginas web como app. Compartilhar contratos OpenAPI e especificações de domínio, não UI/runtime TypeScript com os apps. packages/contracts, domain, authz, api-client, ui-web, config são para backend/web; clientes nativos próprios por plataforma. Gradle e Xcode gerenciam seus projetos. Validar versões estáveis, lockfiles e ADR de migração se já houver código. Logística e portais de rede/frota mantêm áreas web autorizadas próprias; um backend comum para todas as interfaces.

CONTRATOS: API versionada /v1 com OpenAPI e cliente tipado; Supabase Auth para identidade; PostgreSQL/Supabase como fonte de verdade; AuthZ no servidor e RLS/grants no banco para as superfícies acessíveis; Realtime somente em canais autorizados. Negócio e escrita privilegiada no backend; consultas diretas do cliente apenas quando explicitamente aprovadas e protegidas por RLS. Service-role e segredos exclusivamente no servidor, jamais bundles/public env. Roles não podem vir de metadata editável pelo usuário. Multi-tenant por organização e unidade; evitar caches compartilhados entre tenants. Finanças, auditoria e documentos sensíveis em superfícies restritas.

SEGURANÇA: MFA obrigatório para staff privilegiado; cookies/tokens apropriados a cada plataforma, validação server-side, CORS allowlist, CSP e headers na web, rate limits, proteção a CSRF/XSS/SQLi/SSRF/IDOR, uploads seguros, rotação/revogação, log redigido sem segredos/PII indevida. Mobile usa armazenamento seguro de tokens e permissões mínimas. Não armazenar dados de cartão. Endereço/localização/documentos com exposição e retenção mínimas. Mudança bancária, repasses e reembolsos acima da alçada exigem controle adicional. Threat model e checklist OWASP ASVS versionados como metas verificáveis, sem alegar certificação. Revisão jurídica e pentest externo continuam pendências de lançamento.

PADRÕES DE DOMÍNIO: centavos inteiros para BRL; datas persistidas UTC e exibidas America/Sao_Paulo; validação de timezone de horário de loja; UUIDs/IDs não substituem autorização; transações, constraints, índices e idempotency keys para mutações críticas; outbox e consumidores idempotentes para tarefas assíncronas. Máquina de estados e histórico de eventos de pedido separados dos estados de pagamento e entrega. Não chamar provedor

externo dentro de transação longa. Estado final do PSP exige validação de assinatura, correlação e reconciliação. Não confiar em preço, comissão, papel ou status enviado pelo cliente.

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

QUALIDADE: cada módulo traz testes unitários de regras, integração local com Supabase, casos de acesso negado e multi-tenant, contratos e e2e aplicáveis. CI exige lint, typecheck, testes, RLS/pgTAP, build e secret scan. Propor meta inicial de cobertura de 80% em código de domínio e cobertura de decisões críticas, sem manipular métricas; não é resultado garantido. Testes ausentes ou pulados explicitamente relatados. Nunca executar testes com cobrança ou dados sintéticos em produção.

FINAL DE CADA ETAPA: listar arquivos alterados; migrations; endpoints; telas; permissões; comandos executados e resultados reais; cobertura; evidências; riscos; pendências de configuração; atualização de docs e próximos passos. Se faltar credencial/aprovação, continuar apenas no trabalho seguro local e registrar BLOQUEADO, não CONCLUÍDO. Após os testes e secret scan, realizar commit e push no GitHub alvo, respeitando gates do ambiente e confirmando SHA remoto.

PROMPT M00 — Auditoria, produto e contratos compartilhados

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: Início obrigatório.

OBJETIVO: Estabelecer a base comum antes de qualquer app.

IMPLEMENTAÇÃO E TELAS

Inspecione workspace e Git sem ler/imprimir segredos. Produza PRD, escopo por fase, matriz de apps/atores, inventário de integrações, decisões abertas e ADR de stack. Defina fontes de verdade, limites dos módulos, convenções de IDs/dinheiro/tempo/paginação/erros e contrato OpenAPI inicial. Documente que marketplace de comida é o núcleo; mercado, publicidade e assinatura são extensões condicionadas.

Decisões bloqueadoras: projeto Supabase correto; ambiente dev/stage/prod; modalidade loja/plataforma/híbrida; região; PSP/split; mapas; comunicação; política de taxas, cancelamento, repasse e cadastro dos entregadores. Proponha padrões, não invente aprovação.

DADOS, CONTRATOS E ACESSOS

Documentos e schemas de contratos; sem escrita remota ou dados de domínio.

TESTES OBRIGATÓRIOS

Validar schemas e consistência dos contratos; identificar comandos reproduzíveis e ferramentas disponíveis.

M00 — Auditoria, produto e contratos compartilhados

CRITÉRIOS DE ACEITE

Plano com dependências, riscos, critérios de aceite, decisões aprovadas/abertas e nenhum segredo no repositório.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M00.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M01 — Monorepo, ambientes e desenvolvimento local

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M00.

OBJETIVO: Criar estrutura executável para todos os aplicativos.

IMPLEMENTAÇÃO E TELAS

Crie apps e packages da arquitetura aprovada, workspace pnpm, lint/format/typecheck, lockfile, configuração de ambiente validada por app, logging redigido e health endpoints reais. Instale Supabase local e scripts de reset somente local com bloqueio explícito para host remoto. Separar .env por aplicativo; .env.example só nomes/placeholders. Diferenciar build-time/runtime env e variáveis públicas. Crie esqueletos sem telas de demonstração e sem dados falsos. Integrações ausentes devem permanecer indisponíveis.

DADOS, CONTRATOS E ACESSOS

Configuração de ambiente e health, com supabase/seed.sql fictício, idempotente, persistido no Supabase de desenvolvimento/teste e sem dados chumbados nas interfaces. Definir conexão server e client separadas.

TESTES OBRIGATÓRIOS

Build de cada app, lint, typecheck e teste de validação de env sem expor valores; garantir secrets ausentes em bundles.

CRITÉRIOS DE ACEITE

Todos os apps iniciam localmente ou explicam exatamente dependências bloqueadas; setup reproduzível por README.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M01.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Não aplicar mudanças remotas ou

M01 — Monorepo, ambientes e desenvolvimento local

publicar sem consentimento; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M02 — Supabase: banco, migrations, Auth, RLS e Storage

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M00, M01; projeto dwsoochnfxldfvcaijie; executar migrations e seed no Supabase autorizado.

OBJETIVO: Implementar a fundação de dados e isolamento.

IMPLEMENTAÇÃO E TELAS

Desenhe ERD/dicionário para profiles, organizations/members, stores/hours/service_areas, catálogo, endereços, pedidos/itens/eventos, entregas, referências de pagamento, ledger, suporte, promoções e auditoria. Adicione entidades por fase, evitando dependências circulares. snapshots imutáveis do pedido, constraints de valores e estados, índices/geoespaciais se aprovados.

Ative RLS e grants mínimos nas tabelas expostas; por operação e tenant, testando consumidor, parceiro, entregador e staff. Proteja views/RPCs e SECURITY DEFINER com search_path e permissões explícitos. Buckets privados para documentos, políticas, MIME/tamanho, URLs curtas e rastreáveis. Catálogo público somente dados aprovados. Proteja escopo Realtime e subscriptions. Não alterar schema existente sem análise e plano de compatibilidade.

DADOS, CONTRATOS E ACESSOS

Migrations SQL versionadas, contratos de acesso, rollback/forward recovery; execução local antes de aplicar as migrations no Supabase autorizado, executando de fato os SQLs e verificando o resultado.

TESTES OBRIGATÓRIOS

pgTAP allow/deny por tabela/papel/ação, acesso cruzado entre organizações, RPC/view/storage,

constraints e migration from clean local DB.

CRITÉRIOS DE ACEITE

ERD, dicionário, matriz RLS, migrations e evidências locais; nenhuma política allow-all para contornar erros.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M02.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M02 — Supabase: banco, migrations, Auth, RLS e Storage

PROMPT M03 — Identidade, contas, permissões e onboarding

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M02.

OBJETIVO: Autenticar e autorizar todos os aplicativos sem bypass.

IMPLEMENTAÇÃO E TELAS

Implemente cadastro/login/logout/recuperação e verificação de contato via Supabase Auth conforme canais configurados. Convites para equipes com expiração, aceite auditado e escopo organização/unidade. MFA e step-up para funções privilegiadas. Aprovação de loja/entregador não confere papel editável pelo próprio usuário. Onboarding inicial do primeiro admin por procedimento local controlado, nunca endpoint público ou conta/senha fixa. Sessão web segura e tokens mobile em secure storage. Revogar sessão, dispositivos e acessos no desligamento. Fluxos de exportação/exclusão de conta com análise de retenção legal.

DADOS, CONTRATOS E ACESSOS

Profiles, memberships, staff scopes, invitations, approval events; contratos /me, membros e preferências.

TESTES OBRIGATÓRIOS

Login inválido, recuperação, expiração, revogação, tentativa de autoelevação de papel, multi-tenant, MFA staff e deep link mobile seguro.

CRITÉRIOS DE ACEITE

Permissões reais em API e banco; menu oculto não substitui autorização; nenhum login demo.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M03.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M04 — Catálogo, lojas, cobertura e preços

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M02, M03.

OBJETIVO: Criar serviços de negócio reutilizados por cliente e parceiro.

M03 — Identidade, contas, permissões e onboarding

M04 — Catálogo, lojas, cobertura e preços

IMPLEMENTAÇÃO E TELAS

Implemente cadastro de loja/categoria/produto/adicionais, regras de mínimo/máximo, disponibilidade, fotos autorizadas, horário normal/exceção, pausa e raio/polígono de cobertura conforme decisão. Publicação só de loja aprovada e operacional. Busca paginada com filtros reais e consulta geográfica eficiente. Serviço de cotação server-side com preço dos itens, adicionais, taxas, entrega e descontos aprovados. Validar endereço/cobertura/horário na cotação e novamente no pedido; versionar ou expirar cotação. Não inventar tempo de entrega ou taxa sem regra aprovada.

DADOS, CONTRATOS E ACESSOS

/stores, /catalog, /availability, /quotes; catálogo público separado de dados privados.

TESTES OBRIGATÓRIOS

Horário virando dia, feriados, fuso, fora de cobertura, produto pausado, adicionais inválidos, preço modificado, paginação e tenant.

CRITÉRIOS DE ACEITE

Busca/cotação retornam apenas dados reais; sem loja publicada mostrar vazio; preços calculados no servidor.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M04.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M05 — App web do cliente — portal de compra/PWA

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10; pagamento final depende M12.

OBJETIVO: Construir apps/customer-web completo, não só landing page.

IMPLEMENTAÇÃO E TELAS

Telas: entrada/cadastro/recuperação, localização e endereços, descoberta/busca/filtros, detalhe da loja, produto/adicionais/observação, carrinho, cotação e checkout, resultado/pendência de pagamento, acompanhamento e histórico do pedido, avaliação após entrega, perfil, privacidade e suporte.

Carrinho de uma loja por pedido na primeira versão; troca pede confirmação. Totais vêm da cotação real. Mostrar frete, taxas e prazo somente se calculados. Revalidar indisponibilidade ao concluir. Checkout bloqueado se PSP ou política não configurados; não exibir pedido pago artificial. Acompanhamento por eventos autorizados com fallback de consulta. PWA com estratégia de cache que nunca vaza pedidos, endereços ou tokens; pedido/checkout sempre

M05 — App web do cliente — portal de compra/PWA

online e explícito. SEO só nas páginas públicas aprovadas. Acessibilidade de teclado/leitor, formulários e erros.

DADOS, CONTRATOS E ACESSOS

Consumir api-client compartilhado; sem duplicar lógica de preços; /me/addresses, stores/catalog/quotes/orders/support.

TESTES OBRIGATÓRIOS

Componentes e formulário; e2e compra em ambiente de teste real/sandbox configurado; preço adulterado, produto indisponível, repetição de clique, sessão expirada e dados de outro cliente negados.

CRITÉRIOS DE ACEITE

Fluxo completo real quando configurado, com vazio/erro honesto caso contrário;

documentação de telas e e2e.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M05.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M06 — App mobile do cliente — Android e iOS

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10; M12 para pagamento; contratos de M05.

OBJETIVO: Construir apps/customer-android em Kotlin/Jetpack Compose e apps/customer-ios em Swift/SwiftUI, independentes e nativos, sem HTML embutido ou framework híbrido.

IMPLEMENTAÇÃO E TELAS

Android Kotlin/Jetpack Compose e iOS Swift/SwiftUI, implementações independentes com navegação nativa, cadastro/login/recuperação, seleção de endereço, permissão de localização opcional/manual, descoberta, loja/cardápio/adicionais, carrinho/cotação/checkout, pedidos/tracking, perfil/suporte e preferências de notificações. Tokens em armazenamento seguro; deep links allowlist para Auth/pagamento/pedido. Push apenas após consentimento e configuração real, revogável por dispositivo. Offline mostra dados marcados como desatualizados e não conclui compra. Não compartilhar token em URL/log. Configurar IDs, ícones/splash próprios sem imitar marca iFood. Builds separados Android/iOS com assinatura providenciada fora do Git; publicação em lojas exige conta e aprovação do proprietário.

DADOS, CONTRATOS E ACESSOS

api-client/domain compartilhados; UI mobile distinta; callback de pagamento validado no

M06 — App mobile do cliente — Android e iOS

backend.

TESTES OBRIGATÓRIOS

Unitários/componentes, integração API, testes de navegação/deep links, token storage, permissão negada, retomada do app e e2e em dispositivo/emulador disponível.

CRITÉRIOS DE ACEITE

App executável Android/iOS conforme ferramentas disponíveis; resultados reais e instruções de build, assinatura e publicação sem alegar upload não realizado.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M06.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M07 — Portal web do parceiro — gestão do estabelecimento

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10; M13 para financeiro.

OBJETIVO: Construir área de gestão em apps/partner-web.

IMPLEMENTAÇÃO E TELAS

Telas: onboarding/documentos e status, visão geral baseada em métricas reais, cadastro da loja, horários/exceções, área e taxas conforme política, cardápio/categorias/adicionais/fotos, disponibilidade, membros/convites/permissões, avaliações, relatórios, extratos/repasses, condições contratuais e suporte. Separar proprietário, gerente, operador e financeiro. Exportações paginadas/assíncronas autorizadas e auditadas. Alteração de conta bancária não liquida dinheiro e requer step-up/aprovação conforme política. Sem dados, não fabricar vendas/gráficos. Portal acessa somente própria organização/unidades.

DADOS, CONTRATOS E ACESSOS

Módulos de loja/catalog/members/reports; financeiro como leitura de M13, não cálculo duplicado.

TESTES OBRIGATÓRIOS

CRUD real, validação upload, escopo por unidade, convite indevido, financeiro negado para operador e exportação cruzada negada.

CRITÉRIOS DE ACEITE

Parceiro consegue cadastrar e manter operação real; nenhuma visão global de outros parceiros.

M07 — Portal web do parceiro — gestão do estabelecimento

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M07.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M08 — Gestor operacional de pedidos do restaurante

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M07, M10.

OBJETIVO: Construir console de operação dentro do partner-web, separado da gestão.

IMPLEMENTAÇÃO E TELAS

Fila com novo/aceito/em preparo/pronto/retirado/finalizado/cancelado, detalhes e snapshots, alertas sonoros opcionais com consentimento, temporizadores derivados de timestamps reais, aceitar/rejeitar com motivo conforme política, prazo de preparo validado, avisar pronto e confirmar coleta por regra do domínio. Reconexão busca eventos perdidos e evita duplicatas. Conflitos entre operadores tratados por versão/lock otimista. Impressão local explícita; integração impressora/KDS apenas com hardware/API configurados. Pausa da loja e de item reais; pedidos ativos não desaparecem ao pausar. Contato cliente somente necessário e auditável.

DADOS, CONTRATOS E ACESSOS

Commands de pedidos com expected_version/idempotency, subscriptions tenant-scoped, histórico imutável.

TESTES OBRIGATÓRIOS

Aceite simultâneo, transição inválida, reconnect, duplicação de evento, cancelamento concorrente, loja fechada com pedidos ativos e falta de permissão.

CRITÉRIOS DE ACEITE

Pedido real altera estado uma vez; histórico e cliente/entregador recebem atualização autorizada; sem simulação de fila.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M08.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e

comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M08 — Gestor operacional de pedidos do restaurante

PROMPT M09 — Portal de redes, franquias e multiunidades

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M07, M08, M13.

OBJETIVO: Adicionar gestão consolidada sem romper isolamento.

IMPLEMENTAÇÃO E TELAS

Área no partner-web para organização com múltiplas unidades: seletor de unidade, painel consolidado real, usuários por filial/região, catálogo-base e sobrescritas por loja com regras explícitas, horários locais, relatórios por unidade, exportações e trilha de alterações em lote. Não permitir franchising/compartilhamento entre pessoas jurídicas sem modelo e autorização definidos. Operador de filial não recebe dados globais por erro de filtro. Alterações em lote com prévia, confirmação e resultado parcial por unidade.

DADOS, CONTRATOS E ACESSOS

organization/unit scopes; reports agregados só de unidades autorizadas; operações batch auditadas.

TESTES OBRIGATÓRIOS

Permissões filial/região/rede, consolidação financeira sem duplicar pedidos, operação parcial e cache segregado.

CRITÉRIOS DE ACEITE

Comparativos e gestão em lote só com dados reais e permissão explícita; sem esconder falhas parciais.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M09.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M10 — Pedidos, carrinho, cotação e máquina de estados

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04.

OBJETIVO: Implementar o núcleo transacional antes das interfaces dependentes.

M09 — Portal de redes, franquias e multiunidades

M10 — Pedidos, carrinho, cotação e máquina de estados

IMPLEMENTAÇÃO E TELAS

Defina estados separados de pedido, pagamento e entrega; documente tabela de transições/comandos/atores/efeitos. Criar pedido com cotação válida, snapshot de itens/preço/endereço necessário, idempotência e prevenção de duplicidade. Reserva de disponibilidade quando aplicável, rejeição/expiração e compensações. Modalidades retirada, entrega loja ou plataforma apenas se aprovadas. Confirmar pedido comercial e pagamento conforme fluxo escolhido; não pressupor pagamento aprovado ao criar order. Cancelamento parcial/total e elegibilidade calculados pelo backend e revisados conforme regras comerciais. Histórico append-only e outbox transacional para notificações/jobs. Tratar PSP aceito depois de expirar pedido como exceção conciliável, não sucesso silencioso.

DADOS, CONTRATOS E ACESSOS

/quotes, /orders, /orders/{id}/commands, /orders/{id}/events; enums publicados e versões controladas.

TESTES OBRIGATÓRIOS

Testes de transição completa, preço manipulado, idempotência, concorrência, expiração, compensação, timeout e rollback; isolamento usuário/loja.

CRITÉRIOS DE ACEITE

Nenhum estado impossível; efeitos financeiros/logísticos não duplicam; tabela de transições e testes correspondentes.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M10.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M11 — App do entregador — Android e iOS

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M10, M14; M13 para ganhos.

OBJETIVO: Construir apps/courier-android e apps/courier-ios com fluxo logístico real.

IMPLEMENTAÇÃO E TELAS

Telas: cadastro/documentos/aprovação, veículo/região, disponibilidade online/offline, oferta com dados comerciais aprovados, aceite/recusa, navegação até loja, chegada e coleta, rota ao cliente, ocorrência, confirmação por código/evidência conforme política, histórico, ganhos/extrato e suporte em rota. Somente após atribuição expor endereço necessário; cliente não deve receber documento/endereço pessoal do entregador. Ofertas expiram e aceite é transacional. Localização em primeiro e segundo plano apenas quando necessário, com

M11 — App do entregador — Android e iOS

consentimento, finalidade, retenção e permissões Android/iOS. Não prometer background com app fechado sem verificar restrições reais de SO. Usar push configurado e estratégia real de reconexão; sem coordenadas simuladas em runtime. Offline registra estado com timestamp e reenvio idempotente apenas quando seguro; não conclui entrega fictícia. Se GPS negado, explicar impacto e suporte, sem criar rastreamento falso.

DADOS, CONTRATOS E ACESSOS

/couriers/me, availability, offers, assignments, locations, delivery commands, earnings; RLS e API só para atribuições permitidas.

TESTES OBRIGATÓRIOS

Aceite simultâneo, oferta expirada, permissão GPS negada, rede instável, timestamp antigo, tentativa de acessar outra rota, código inválido, duplicação de entrega e saldo real.

CRITÉRIOS DE ACEITE

Entrega rastreável com evidências autorizadas e ganhos reconciliados; builds e permissões documentados, pendências de dispositivo explícitas.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M11.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com

motivo, nunca simulação.

PROMPT M12 — Pagamentos, PSP, webhooks e estornos

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.
DEPENDÊNCIAS: M10; escolha e credenciais seguras do PSP.
OBJETIVO: Implementar cobrança real com limites claros por provedor.

IMPLEMENTAÇÃO E TELAS

Primeiro registrar ADR de PSP: Pix/cartão, split 1:1/1:N efetivamente contratado, onboarding/KYC, liquidação, taxas, sandbox, refund/chargeback, webhooks e conciliação. Não assumir split para entregador se serviço não oferece. Usar SDK/tokenização/checkout do PSP, sem PAN/CVV no servidor ou log. Integração runtime real; provider ausente impede cobrar. Sandbox identificado é permitido em ambiente de testes, não como pagamento de produção. Assinar/verificar webhook e replay window conforme documentação do PSP; correlacionar merchant/pedido/valor/moeda; idempotência e eventos fora de ordem; não confiar no redirect do browser para marcar pago. Reconciliar por consulta ao PSP e backoffice. Proteger segredo, usar retries com backoff e dead-letter. Refund parcial/total sujeito a alçada, idempotência e ledger, com estados pendentes/falhos. Evitar saldo virtual que alegue custódia não contratada.

M12 — Pagamentos, PSP, webhooks e estornos

DADOS, CONTRATOS E ACESSOS

payment_attempts, provider_events, refunds; referências/metadata mínimas e sem dados de cartão.

TESTES OBRIGATÓRIOS

Unitários com doubles só em testes; integração sandbox real quando credenciais disponíveis; assinatura inválida, replay, timeout pós-cobrança, duplicate webhook, chargeback, refund repetido e valor divergente.

CRITÉRIOS DE ACEITE

Pagamentos e estornos com estado/evidência verificável; pendências sem credencial marcadas BLOQUEADO, não sucesso.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M12.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado,

respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M13 — Backoffice financeiro e conciliação

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.
DEPENDÊNCIAS: M10, M12; M14 para entregas.
OBJETIVO: Construir área financeira própria em admin-web e leituras autorizadas nos portais.

IMPLEMENTAÇÃO E TELAS

Telas: transações, ledger, comissões, fretes, subsídios/cupons por financiador, recebíveis por estabelecimento, ganhos de entregador, repasses previstos/executados, conciliação PSP, divergências, refund/chargeback e exportação. Regras financeiras versionadas por contrato e vigência. Ledger de partidas dobradas com lançamentos balanceados, imutáveis e reversões auditadas; valores em centavos. Não calcular receita como total bruto sem distinguir repasses. Payout somente por provedor/contrato confirmado e autorização com segregação de função; nenhum botão pode simular transferência. Fechamentos e relatórios explicam corte de data, moeda, status e origem. Não substituir contabilidade oficial nem emitir documento fiscal sem solução aprovada.

DADOS, CONTRATOS E ACESSOS

ledger accounts/journals/entries, settlements, payout references, reconciliation runs/matches; acesso financeiro restrito.

TESTES OBRIGATÓRIOS

Balanceamento, reversão, replay, diferença de centavos, conciliação parcial, comissão de

M13 — Backoffice financeiro e conciliação

vigências distintas, permissão e aprovação de payout; leitura operador negada.

CRITÉRIOS DE ACEITE

Todo total tem trilha até transação real; divergências visíveis; nenhuma transferência sem confirmação do PSP.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M13.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com

motivo, nunca simulação.

PROMPT M14 — Logística, despacho e console operacional

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.
DEPENDÊNCIAS: M10, M03; modelo de entrega aprovado.
OBJETIVO: Implementar backend logístico e console no admin-web.

IMPLEMENTAÇÃO E TELAS

Telas: pedidos aguardando coleta, entregadores disponíveis reais, mapa somente com integração e coordenadas reais, fila sem entregador, atribuir/reofertar/reatribuir com motivo, atrasos, ocorrências e histórico. Diferenciar entrega própria da loja e entrega plataforma; sem rastreio, não alegar posição real. Calcular rota/cotação/ETA via provedor escolhido ou regra simples explicitamente aprovada; nunca inventar ETA. Despacho manual primeiro; automático após política aprovada de elegibilidade/distância/capacidade/expiração. Uma entrega não pode ter duas atribuições ativas indevidas. Roteirização múltipla depende de política de prazo/capacidade, não habilitar por padrão. Geolocalização com retenção curta configurada, janela operacional e controles antiacesso indevido; incerteza de GPS tratada, não tomada como prova única.

DADOS, CONTRATOS E ACESSOS

delivery_tasks, offers, assignments, location samples, delivery events; eventos internos e app M11.

TESTES OBRIGATÓRIOS

Atribuição concorrente, entregador offline, expiração, reatribuição após coleta bloqueada ou controlada, coordenada antiga, recálculo e acesso restrito.

CRITÉRIOS DE ACEITE

Despacho real auditado; atrasos/sem cobertura visíveis; sem mapa ou motorista fictício.

M14 — Logística, despacho e console operacional

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M14.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M15 — Portal de operadores logísticos e frotas

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M11, M14, M13; contratos de frota aprovados.

OBJETIVO: Disponibilizar área restrita de gestão de frota, se modelo contratar frotas.

IMPLEMENTAÇÃO E TELAS

Criar área no admin-web ou portal dedicado somente por ADR, com organização de frota, gestores, entregadores vinculados/autorizados, disponibilidade, entregas delegadas, ocorrências, desempenho e extratos contratuais. Não permitir que gestor de frota leia todos os entregadores independentes nem pedidos de outra frota. Vínculo/saída de entregador com consentimento/documentação e auditoria; ganhos e payout seguem contrato aprovado, não supor vínculo trabalhista ou repasse à frota. Se modelo de frota não aprovado, documentar DESABILITADO, sem telas falsas.

DADOS, CONTRATOS E ACESSOS

fleet organizations/members/contracts e assignments delegadas, respeitando limites de tenant.

TESTES OBRIGATÓRIOS

Frota A versus B, entregador desvinculado, pedido não delegado, extrato e autorização de gestor.

CRITÉRIOS DE ACEITE

Área habilitada só para modelo real contratado; escopo e responsabilidades documentados.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M15.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M15 — Portal de operadores logísticos e frotas

PROMPT M16 — Administração central, acessos e cadastros

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10.

OBJETIVO: Construir a estrutura do backoffice em apps/admin-web.

IMPLEMENTAÇÃO E TELAS

Dashboard por papel com métricas reais e definição dos indicadores. Módulos de usuários, organizações, lojas, entregadores, documentos/aprovações, contratos/condições, bloqueios/suspensões motivados, filas de operação, pedidos e histórico. RBAC com escopo regional/unidade quando aprovado e negativa por padrão. Não usar superadmin como conta diária. Contas individuais, convites, MFA, expiração/revogação e auditoria de ações sensíveis. Proibir impersonation silenciosa; se implementada após ADR, consentimento/regra, TTL, banner e log explícitos. Busca por dados pessoais limitada e rastreável; exportação com escopo/alçada. Auditoria não editável por operador comum.

DADOS, CONTRATOS E ACESSOS

/admin/users, merchants, couriers, approvals, access, audit; finanças e logística continuam módulos próprios.

TESTES OBRIGATÓRIOS

Matriz de papéis, tentativa de grant próprio, alteração de dado bancário, ação sem justificativa, exportação sem permissão e busca cross-tenant não autorizada.

CRITÉRIOS DE ACEITE

Cada menu corresponde a endpoint autorizado; papéis separados para operação, suporte, financeiro, comercial, marketing, risco, auditoria e técnico.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M16.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M17 — Atendimento, suporte e resolução de incidentes

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

M16 — Administração central, acessos e cadastros

M17 — Atendimento, suporte e resolução de incidentes

DEPENDÊNCIAS: M10, M16; M12/M13 para reembolso.

OBJETIVO: Construir console de atendimento com escopo e alçadas.

IMPLEMENTAÇÃO E TELAS

Telas de ticket, fila/prioridade, contexto do pedido, timeline, participantes, mensagens, anexos privados, categorização, responsável, SLA configurado, escalonamento e conclusão. Entrada de suporte no cliente, parceiro e entregador; entregador em rota priorizado conforme regra aprovada. Canais externos só quando configurados. Atendente vê dados necessários e pode solicitar ação; refund/payout segue fluxo financeiro e alçada, não edição direta de estado. Registro de autoria, motivo e evidências; anonimização/retenção conforme política. Não enviar mensagens reais a destinatário sem ação autorizada e confirmação adequada na operação.

DADOS, CONTRATOS E ACESSOS

support_cases/messages/attachments, action_requests e auditoria; acesso por participante/atribuição e staff autorizado.

TESTES OBRIGATÓRIOS

Acesso a ticket alheio, arquivo malicioso, reembolso acima da alçada, ação repetida, SLA e mascaramento de PII.

CRITÉRIOS DE ACEITE

Chamados reais integrados aos apps e ações rastreáveis; histórico preservado sem exposição indevida.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M17.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M18 — Comercial, contratos e onboarding de parceiros

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M16, M07.

OBJETIVO: Separar prospecção comercial do financeiro e da operação.

IMPLEMENTAÇÃO E TELAS

Área comercial: leads cadastrados legitimamente, pipeline, responsáveis, contato com

base/finalidade documentada, proposta, condições e vigência, aprovação de comissão/desconto excepcional e conversão para organização/loja após aceite. Contrato não é válido só por clicar em botão: integração de assinatura ou aceite com evidências precisa de aprovação jurídica.

M18 — Comercial, contratos e onboarding de parceiros

Comercial não pode executar repasse nem acessar documento além do necessário. Não carregar lista comprada/fictícia nem realizar disparos externos automaticamente.

DADOS, CONTRATOS E ACESSOS

leads/proposals/contracts/rule versions/approvals, relação com organização sem duplicar cadastro.

TESTES OBRIGATÓRIOS

Condição sem aprovação, vigência futura, acesso financeiro negado e conversão duplicada.

CRITÉRIOS DE ACEITE

Condições auditadas e versionadas; contrato/template pendente de jurídico claramente sinalizado.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M18.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M19 — Marketing, cupons, avaliações e campanhas

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M04, M10, M13, M16.

OBJETIVO: Implementar promoções com impacto financeiro explícito.

IMPLEMENTAÇÃO E TELAS

Criar campanhas e cupons com elegibilidade, validade/fuso, limite por cliente/pedido/orçamento, valor mínimo, teto e financiador (plataforma/loja) aprovados. Aplicação atômica para evitar ultrapassar orçamento por concorrência. Relatório por resgate real, não alcance inventado. Avaliação somente de pedido elegível e autor autenticado, uma avaliação por política, moderação rastreável e resposta do parceiro; nada de estrelas iniciais falsas. Segmentação minimizada/consentimento conforme base jurídica; comunicação externa

só via fornecedor configurado. Publicidade patrocinada e assinatura apenas após contratos/preço aprovados, com identificação transparente, não implementar cobrança fictícia.

DADOS, CONTRATOS E ACESSOS

promotions, redemptions/reservations, reviews/moderation, campaigns e ledger de subsídio.

TESTES OBRIGATÓRIOS

Resgate simultâneo, cupom expirado, orçamento, cancelamento/reversão, elegibilidade, autoavaliação indevida e permissão de marketing.

M19 — Marketing, cupons, avaliações e campanhas

CRITÉRIOS DE ACEITE

Desconto refletido na cotação, pedido e conciliação; métricas reais e comunicações configuradas.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M19.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M20 — Notificações, tempo real e integrações externas

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.
DEPENDÊNCIAS: M03, M10; eventos dos demais módulos.
OBJETIVO: Centralizar comunicação confiável e rastreável.

IMPLEMENTAÇÃO E TELAS

Implemente outbox transacional/worker, templates versionados por evento, preferências e consentimento aplicável; canais in-app e push, e-mail/SMS/outros somente com provedor aprovado. Realtime não substitui armazenamento durável: reconectar e recuperar eventos por cursor. Filas com idempotência, backoff, limites, dead-letter e replay controlado. Revogar device tokens, diferenciar iOS/Android/web, não expor tokens ao parceiro. Conteúdo mínimo na tela bloqueada. ERP/PDV/iFood: conectores somente se contratados e homologados; não são dependência do marketplace próprio. Webhooks externos com assinaturas, tenant e logs redigidos. Não ativar loop de mensagens em teste ou disparo para clientes reais.

DADOS, CONTRATOS E ACESSOS

outbox, notification preferences/deliveries/devices, connector configs em secret store e eventos correlacionados.

TESTES OBRIGATÓRIOS

Duplicatas, falha/retry, token revogado, canal sem credencial, tenant leakage, webhook inválido e replay sem reenvio indevido.

CRITÉRIOS DE ACEITE

Entrega registrada de forma honesta: enfileirado/enviado/confirmado/falhou conforme evidência do provedor, sem inventar confirmação.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M20.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e

M20 — Notificações, tempo real e integrações externas

comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M21 — Relatórios, indicadores, risco e privacidade

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M13, M16, M17, M19.

OBJETIVO: Criar relatórios confiáveis e rotinas de governança.

IMPLEMENTAÇÃO E TELAS

Relatórios: volume de pedidos, GMV versus receita líquida claramente definidos, conversão somente com eventos reais e consentimento aplicável, taxa de cancelamento com denominador, prazo de preparo/entrega com timestamps, disponibilidade, SLA e margens conforme custos registrados. Dashboard sem dados mostra vazio. Exportações auditadas com expiração e acesso restrito; agregação não pode reidentificar indevidamente. Risco: regras explicáveis e revisão humana de bloqueio, sem alegar IA antifraude inexistente.

Privacidade: inventário de dados por finalidade/base a validar com jurídico, acesso/portabilidade/correção/exclusão, retenção por categoria, bloqueio de exclusão quando retenção legal aplicável, anonimização, incidentes e contatos. Auditor/encarregado vê somente o necessário; consentimento não usado como base genérica para tudo.

DADOS, CONTRATOS E ACESSOS

queries/materialized views autorizadas, analytics events minimizados, privacy requests, retention jobs e risk decisions.

TESTES OBRIGATÓRIOS

Fórmulas com conjuntos de teste isolados, fuso/corte de data, dados faltantes, exportação negada, retenção e anonimização sem destruir trilha financeira necessária.

CRITÉRIOS DE ACEITE

Cada KPI tem fórmula/fonte/período; relatório reproduzível; políticas jurídicas pendentes explicitadas.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M21.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M21 — Relatórios, indicadores, risco e privacidade

PROMPT M22 — Extensão mercados, separação e substituições

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M04, M10, M12, M07; só após escopo aprovado.

OBJETIVO: Cobrir supermercado sem distorcer o núcleo inicial.

IMPLEMENTAÇÃO E TELAS

Implementar área de separação no partner-web: lista por pedido, unidade/peso, indisponibilidade, proposta de substituição, consentimento do cliente, atualização de preço e reflexo financeiro. Definir autorização/captura/reembolso compatível com PSP antes de alterar cobrança. Catálogo com unidade de medida e precisão decimal de quantidade, dinheiro em centavos; proibir ajuste arbitrário. Recusa/timeout de substituição seguem política aprovada. Não habilitar medicamento, álcool ou outras categorias reguladas sem revisão específica. Se extensão não aprovada, mantê-la documentada e desabilitada.

DADOS, CONTRATOS E ACESSOS

pick tasks/items, substitution proposals/approvals e ajustes vinculados a pedido/pagamento.

TESTES OBRIGATÓRIOS

Pesagem, arredondamento, substituição recusada/expirada, cobrança insuficiente, reembolso parcial e concorrência.

CRITÉRIOS DE ACEITE

Quantidade/preço final com consentimento e conciliação; sem prometer ajuste PSP não suportado.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M22.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M23 — Cibersegurança transversal e auditoria técnica

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M01 em diante; gate antes do release.

OBJETIVO: Executar hardening verificável em todos os apps.

M22 — Extensão mercados, separação e substituições

M23 — Cibersegurança transversal e auditoria técnica

IMPLEMENTAÇÃO E TELAS

Produzir threat model de fronteiras web/mobile/API/banco/workers/PSP, inventário e checklist ASVS com versão e IDs verificados na documentação, requisito/evidência/status/risco. Revisar auth/MFA, RLS, endpoints, upload, SSRF/IDOR/injection, CSRF/CSP/CORS, secure storage mobile, deep links, secrets, dependências/SBOM, log redaction, isolamento cache/Realtime, rate limits distribuídos quando necessários, backup criptografado e incident response. CI com secret scan, SAST/dependency audit e análise de containers se usados. Não escanear infraestrutura externa sem autorização. Achado crítico/alto bloqueia release até resolução ou aceite formal de risco, não simples comentário. Risco residual deve ficar explícito; não alegar certificação, invulnerabilidade ou pentest não feito.

DADOS, CONTRATOS E ACESSOS

docs/security/threat-model.md, ASVS matrix, vulnerability register, disclosure/incident runbooks.

TESTES OBRIGATÓRIOS

Negativos cross-tenant de ponta a ponta, secrets em bundles/logs, replay, privilégio indevido, headers e sessão; testes autorizados em ambiente isolado.

CRITÉRIOS DE ACEITE

Relatório de achados com evidências reais, correções, riscos restantes e checklist por módulo.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M23.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M24 — QA, testes unitários, integração, E2E e desempenho

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: Cada módulo; integração final após M23.

OBJETIVO: Criar estratégia e suíte transversal com gates reais.

IMPLEMENTAÇÃO E TELAS

Unitários de domínio obrigatórios: preço/adicionais/frete/cupom, elegibilidade, horário, estados, idempotência, comissão/ledger, autorização e validação. Componentes web/mobile, integração API/Supabase local real e pgTAP RLS por operação. Contratos OpenAPI/clientes e migrações. E2E: cadastrar loja, catálogo, compra, aceite, preparo, atribuição, coleta, entrega, pagamento sandbox/reconciliação, refund e suporte; separar cenários que dependem de decisões/fornecedores. Segurança, acessibilidade, retry/offline, concorrência e regressão. Testes de carga apenas em ambiente autorizado; definir SLO/carga-alvo com proprietário antes

M24 — QA, testes unitários, integração, E2E e desempenho

de prometer capacidade. Não fakear teste faltante; guardar relatórios/cobertura e testes pulados com motivo. Banco de teste descartável com trava contra execução em produção.

DADOS, CONTRATOS E ACESSOS

test matrix requisito->teste->comando->resultado, pipelines, datasets somente de teste isolados.

TESTES OBRIGATÓRIOS

Executar suítes de verdade e reportar exit codes, artefatos, cobertura por módulo, flakiness e limitações de emulador/PSP.

CRITÉRIOS DE ACEITE

CI falha quando regra crítica falha; sem coverage inflado por exclusões arbitrárias; evidências reproduzíveis.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M24.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M25 — DevOps, observabilidade, deploy, backup e operação

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M01, M23, M24; ambiente/provedores aprovados.

OBJETIVO: Preparar produção sem publicar silenciosamente.

IMPLEMENTAÇÃO E TELAS

Configurar pipeline de lint/typecheck/test/build/scan, proteção de branches recomendada, previews sem segredos prod e ambientes separados. CI/CD para web/API/worker e builds mobile; deployment provider apenas após decisão. Staging primeiro; migrations compatíveis expand/contract, backup pré-mudança e plano de recuperação, aprovação antes de prod. Liveness/readiness reais sem detalhes secretos, logs estruturados com correlation_id e redaction, métricas de erro/latência/filas/webhooks, alertas e tracing sem PII indevida. Definir RPO/RTO e retenção com proprietário; testar restauração em ambiente separado sem sobrescrever prod. Runbooks indisponibilidade Supabase/PSP, fila parada, webhook perdido, vazamento, conta privilegiada comprometida e rollback. Banco único informado não autoriza tratar produção como sandbox.

DADOS, CONTRATOS E ACESSOS

infra versionada sem segredos, workflows, config references, dashboards por backend efetivamente contratado.

M25 — DevOps, observabilidade, deploy, backup e operação

TESTES OBRIGATÓRIOS

Build clean, smoke staging, migration rehearsal, restore drill e rollback; se infraestrutura não contratada, documentar bloqueio.

CRITÉRIOS DE ACEITE

Deploy só após consentimento; checklist de acesso/dominio/TLS/secrets/backups; logs e alertas operacionais reais.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M25.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

PROMPT M26 — Documentação completa, integração final e GitHub

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: Todos os módulos habilitados; M23-M25.

OBJETIVO: Entregar produto integrado com documentação e publicação aprovada.

IMPLEMENTAÇÃO E TELAS

Completar README e docs: PRD/escopo, mapa dos apps e rotas, instalação/env sem segredos, arquitetura/ADRs, ERD/dicionário/migrations, Auth/RLS/permissões, OpenAPI/exemplos sem dados reais sensíveis, manual cliente web/mobile, entregador, parceiro/gestor/rede, admin/backoffice/logística/frota, pagamentos/ledger/conciliação, marketing/suporte, privacidade/segurança/ASVS, testes/CI, deploy/rollback/backup/restore, operação/incidentes/troubleshooting, changelog/contribuição/licença pendente. Diferenciar implementado, configurado, testado, bloqueado e adiado.

Executar fluxo ponta a ponta no ambiente isolado; verificar contratos entre apps e não duplicação de regras. Secret scan no working tree/staged e histórico relevante sem imprimir segredos; achado implica parar e rotacionar, limpeza de histórico requer plano/aprovação.

Verificar remote/branch e preparar resumo dos arquivos, mudanças, testes e limitações. Propor commits pequenos; executar commit/push após testes aprovados, respeitando aprovações do ambiente e publicar somente no repo alvo confirmado, sem force-push e sem mudar visibilidade. Falta de auth GitHub: pedir conexão/credential manager seguro, nunca token no chat. Não dizer publicado sem confirmação do push e branch remoto.

DADOS, CONTRATOS E ACESSOS

docs/DELIVERY_STATUS.md e RELEASE_CHECKLIST com evidência por módulo, URLs somente quando realmente disponíveis.

M26 — Documentação completa, integração final e GitHub

TESTES OBRIGATÓRIOS

Checklist de documentação versus implementação, comandos reproduzíveis, validação de links internos, suite final e repo sem segredos.

CRITÉRIOS DE ACEITE

Release só quando gates cumpridos e pendências aceitas; componentes não configurados não podem ser chamados de sistema completo pronto para produção.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M26.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.